

DHWANI

Working Principles, Technical Specs & What To Do Next

COMPLETE GUIDE & ENGINEERING ROADMAP

Project Name:	Dhwani (AI Voice Agent Suite)
Document Type:	Architecture Walkthrough & Development Backlog
Prepared By:	Antigravity AI Assistant
Target Audience:	Vatshal, Engineering Teams, Reviewers
Status:	Version 1.0 (WebSocket Core Operational)
Next Milestones:	VAD barge-in, Server STT, Telephony Trunks

This document walks you through the active codebase, details the real-time pipeline, and lists actionable future features (with system changes) to build a production-grade voice AI agent that matches market leaders like Vapi, Retell AI, and Bland AI.

1. UNDER-THE-HOOD: HOW DHWANI WORKS TODAY

Dhwani is currently built as a light, high-performance duplex voice pipeline that utilizes WebSockets to coordinate real-time audio playback and user transcription turns. This architecture consists of:

1.1 Frontend Client-Side (Browser Context)

Built in React, the UI handles state changes (dialing, connected, listening, thinking, speaking) and manages browser hardware APIs. It uses the browser's native Web Speech API (`window.SpeechRecognition`) to capture user speech. When the user speaks, the browser transcribes the audio locally to plain text and sends the final string to the backend. This saves server CPU and bandwidth. To play the agent's voice, the frontend decodes base64-encoded MP3 chunks received from the WebSocket into a dynamic HTML5 Audio element.

1.2 Backend Server-Side (FastAPI Context)

Built as an asynchronous FastAPI python backend, the server exposes a WebSocket endpoint `/ws/call`. It maintains session configurations (system prompts, provider choices, voices) and holds the ongoing conversation history list in memory. When it receives text from the frontend, it queries the configured LLM (Google Gemini or Groq Cloud) and feeds the text response into `edge-tts`. `edge-tts` generates an MP3 byte stream using Microsoft's realistic neural voices. The backend base64-encodes this audio and pushes it back to the client.

1.3 Echo Prevention Mechanism

A critical engineering challenge in browser-based voice systems is acoustic loopback (the microphone picking up the agent's voice and transcribing it). Dhwani solves this by programmatically pausing the `SpeechRecognition` listener on the frontend immediately before the HTML5 Audio element starts playing. The frontend hooks into the audio's 'onended' callback. Once the agent is done speaking, the listener is resumed, opening the mic for the user.

2. WHAT TO DO NEXT: CORE FEATURE ENHANCEMENTS

To take Dhwani from a local testing simulator to a production-grade commercial platform like Retell AI, Vapi, or Bland AI, the following modules need to be implemented next. These are ordered by engineering priority:

Priority 1: Voice Activity Detection (VAD) & Barge-in Handling

- **The Problem:** Currently, if the agent is speaking and the user starts talking, the agent cannot be interrupted. The user must wait until the agent finishes playing, which ruins conversational flow.
- **The Solution:** Implement Client-Side or Server-Side VAD (e.g., using silero-vad or WebRTC VAD). If the user starts speaking, the frontend must immediately fire a 'mute/interrupt' event, stop the active HTML5 Audio playback, clear the audio queue on both ends, and switch back to listening state.

Priority 2: Transition to Binary Audio Streaming (WebRTC / Raw Audio)

- **The Problem:** Relying on browser-native SpeechRecognition is inconsistent because text finalization has latency, and it requires specific browsers. It does not work well in silent or noisy backgrounds.
- **The Solution:** Capture raw mic audio chunks via MediaRecorder API or Web Audio API, encode them into low-latency format (like PCM or Opus), and stream them as binary chunks over the WebSocket. On the backend, process these streams using a fast STT engine like Whisper (via Groq/OpenAI) or Deepgram.

Priority 3: Ultra-Low-Latency Voice Engines

- **The Problem:** edge-tts is free, but it takes 800ms-1200ms to synthesize speech because it is a wrapper that waits for the full text before streaming the bytes.
- **The Solution:** Integrate Cartesia (Sonic-beta) or ElevenLabs (Turbo v2) streaming APIs. By sending LLM output chunks to these providers as they generate, we can get audio bytes back in under 150ms, resulting in realistic conversation.

3. ADVANCED ENHANCEMENTS: PRODUCTION ARCHITECTURE

Beyond core voice loops, production deployment requires background storage, telemetry, and external systems integration:

3.1 Telephony Integrations (Real Phone Calls)

Currently, calls are browser-simulated. To make and receive real phone calls, you should configure Twilio Media Streams or Telnyx SIP trunks. In this design, Twilio makes/receives the PSTN call, opens a WebSocket directly to your FastAPI server, and streams bidirectional u-law (PCMU) 8kHz audio. The FastAPI server processes this binary steam, routes it to Whisper/Deepgram STT, queries the LLM, and streams PCMU back to Twilio.

3.2 RAG (Retrieval-Augmented Generation) & Knowledge Bases

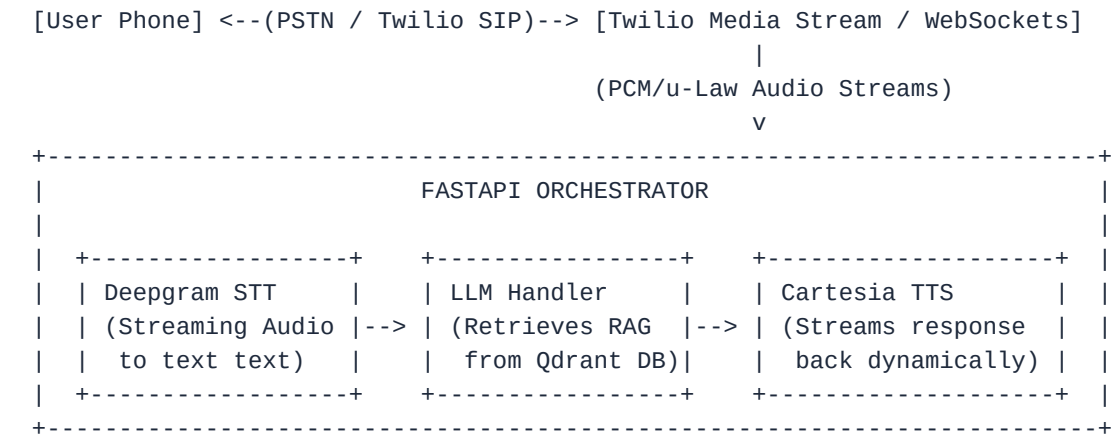
For agents to talk about specific businesses (e.g., dentist schedules, e-commerce orders, product manuals), they need access to external documents. You should integrate a Vector Database (like Qdrant, Pinecone, or pgvector) and build a RAG pipeline. When the user asks a question, the backend performs a semantic search on the vector DB, retrieves the relevant context, and appends it to the LLM system prompt so the model responds with accurate information.

3.3 Persistent Database Schema (SQLite/PostgreSQL)

Currently, configuration, keys, and transcripts exist only in memory and localStorage. You should add database tables to persist:

- Agents Table:	Stores agent profiles (id, name, voice_id, temperature, system_prompt, greeting, creator_id).
- Calls Table:	Logs call details (call_id, agent_id, start_time, duration, status, transcription_log_json, cost).
- API Keys Table:	Saves user credentials securely using encryption at rest.

3.4 Visual Architecture Design of Production System



4. ACTIONABLE STEP-BY-STEP DEVELOPER CHECKLIST

Here is the exact task list to start building out the next phases. Use this checklist as your roadmap:

Phase 1: Environment & Tool Setup (Immediate)

- **Step 1.1:** Create a new git branch 'feature/vad-and-audio-stream' to isolate changes.
- **Step 1.2:** Add database requirements to requirements.txt (SQLAlchemy, alembic, greenlet).
- **Step 1.3:** Configure a local SQLite database and write database initialization models.

Phase 2: Code Refactoring - WebSocket Upgrades

- **Step 2.1:** In app/main.py, change WebSocket receiver from handle_text to accept binary packets.
- **Step 2.2:** Integrate PyAudio or sounddevice on a local debug test script to verify raw recordings.
- **Step 2.3:** Implement a client-side audio capture loop in App.jsx using MediaRecorder API.

Phase 3: Interruption (Barge-In) Logic

- **Step 3.1:** Add 'interrupt' packet type in the WebSocket communication protocol.
- **Step 3.2:** In App.jsx, add an listener that monitors when input volume crosses a noise threshold.
- **Step 3.3:** On noise threshold trigger, call audioRef.current.pause() and emit 'interrupt' over WS.

Phase 4: Deepgram & Cartesia Integration

- **Step 4.1:** Sign up for Deepgram and Cartesia APIs, and add credentials in your backend .env file.
- **Step 4.2:** Replace browser STT with Deepgram Streaming STT inside backend app/main.py WebSocket loop.
- **Step 4.3:** Replace edge-tts with Cartesia streaming inside app/tts.py to achieve sub-200ms latency.

Summary Note:

By following this guide, you will transition Dhvani from a simple sequential text/voice exchange prototype into a state-of-the-art, low-latency, conversational voice agent capable of handling interruptions, real phone lines, and production-level scale.